# GilsonEthernet Assembly Documentation

| Revision History | | |
|---|---|---|
| **Rev.** | **Summary of Change** | **Date** |
| 01 | Original | 1/18/10 |
| | | |
| | | |



**Gilson, Inc. World Headquarters**
3000 Parmenter Street | P.O. Box 620027 | Middleton, WI 53562-0027, USA | Tel: 608-836-1551 | Fax: 608-831-4451
**Gilson S.A.S.** | 19, avenue des Entrepreneurs | BP 145, F-95400 VILLIERS LE BEL, France
**www.gilson.com** | sales@gilson.com | service@gilson.com | training@gilson.com

# Table of Contents

# 1 Assumptions

This Document assumes the user has knowledge of XML, TCP/IP network protocols and Microsoft .NET development experience.

# 2 Overview and Purpose

This document explains how to communicate with and control Gilson Ethernet instruments, including connecting to the instrument, sending commands and receiving responses. All communication is performed over TCP/IP & UDP. This document details how to use the Gilson Ethernet assembly to communicate with devices.

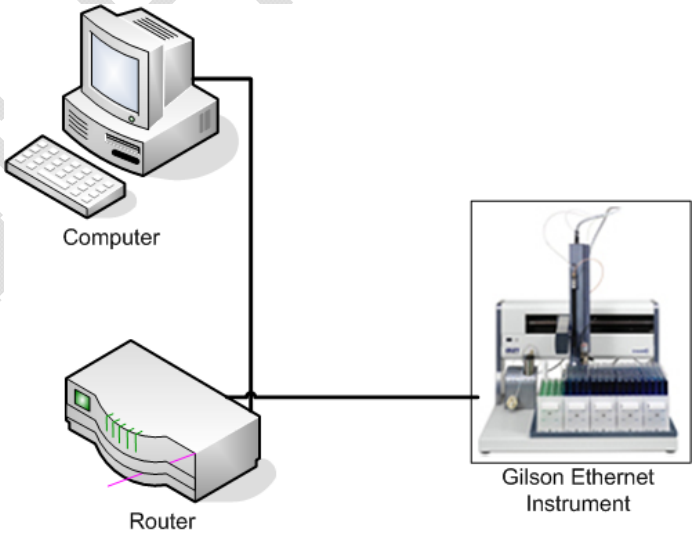
# 3 Definitions

| Synonym | Description |
|---|---|
| Beacon Data | Data broadcasted by the device at a periodic rate. Beacon Data contains XML data describing Ethernet instruments available for control. |
| Communication Port | Port information provided by the device in response to the connection request. This port number is determined by the device and can vary. |
| Instruction Set | The set of commands and parameters that can be used to operate the Device. |

# 4 Gilson Ethernet Instrument Setup

Communication with a Gilson Ethernet Device is established over TCP/IP. Gilson Ethernet Devices broadcast an XML message via UDP called a beacon which is used for device discovery. The beacon data contains information needed to establish communication with the Device as well as available instruments at the specified TCP/IP address. IP address assignment on the instrument side is accomplished via DHCP.

Below is a typical configuration for connecting a PC to a Gilson Ethernet instrument. The router in the image is used to provide a DHCP server on the network which is required by the Gilson Ethernet instrument.




**Ethernet Device Setup**

# 5 Communication with Gilson Ethernet Instruments

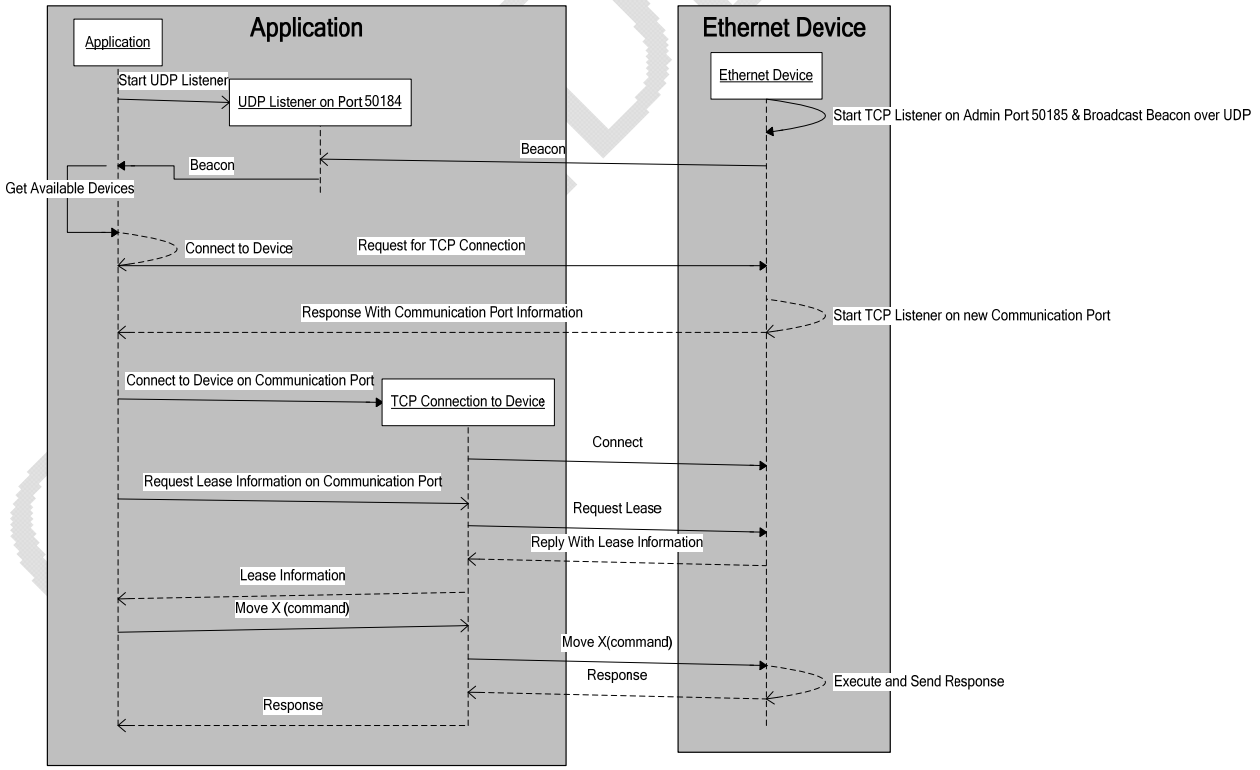
Communication between the instrument and an application is accomplished by sending and receiving XML messages.

Gilson Ethernet instruments broadcast an XML beacon on UDP port 50184 at a periodic rate (typically every two seconds, though this rate can vary for specific instruments). For sample beacon XML, see *Beacon Data Sample XML*.

Discovery of instruments is performed by setting up a UDP listener on port 50184 and monitoring any messages received. The beacon contains information about any available instruments at the TCP/IP address specified.

The following are the steps required to successfully connect to, and communicate with, a Gilson Ethernet instrument:

- Start UDP Listener on port 50184.
- Parse incoming beacon to select the instrument to connect to. Once an instrument has been selected, use the IP address from the beacon to initiate the socket connection.
- Send connection request to Ethernet device on port 50185 and receive back the port to connect to.
- Establish a TCP connection to the instrument using the IP address from the beacon and the port provided in the response to the connection request.
- Determine if a lease or administrative privileges are desired. Depending on the desired functionality, send either the "Renew Lease" or "Admin" command.
- At this point, the instrument is ready to accept and execute commands.




**Sequence Diagram showing communication with Ethernet Device**

```
<Gilson>
  <Beacon>
    <DeviceInfo>
      <IPAddress>192.168.0.100</IPAddress>
      <SubnetMask>255.255.255.0</SubnetMask>
      <DefaultGateway>192.168.0.1</DefaultGateway>
      <MAC>00:40:9d:ba:db:ad</MAC>
      <Instruments>
        <Instrument>
          <Name>System</Name>
          <InstrumentName>System</InstrumentName>
          <ID>-1</ID>
          <SerialNumber>N99999999</SerialNumber>
        </Instrument>
        <Instrument>
          <Name>GX-27x</Name>
          <InstrumentName>GX 271 </InstrumentName>
          <PromVersion>2.2.2.0</PromVersion>
          <ID>35</ID>
          <SerialNumber>260H6N080</SerialNumber>
        </Instrument>
        <Instrument>
          <Name>GX Solvent System</Name>
          <InstrumentName>GX Solvent System</InstrumentName>
          <PromVersion>
          </PromVersion>
          <ID>8</ID>
          <SerialNumber>Serial number could not be received from
device</SerialNumber>
        </Instrument>
      </Instruments>
    </DeviceInfo>
  </Beacon>
</Gilson>
```

**Beacon Data Sample XML**

## 5.1 Receiving Beacon Data

Beacon data can be received from the instrument by starting the UDP listener on BeaconPort 50184. The application can specify a specific port number or use the GilsonPorts class exposed by the Gilson Ethernet assembly. The GilsonPorts class defines a constant for BeaconPort. UDP Listener is represented by the class EthernetUDPListener.

Sample code for starting UDP listener:

```
EthernetUDPListener _udpListner = new EthernetUDPListener(
     new EthernetDataDelegate(BeaconData), GilsonPorts.BeaconPort);
_udpListner.Start();
```

**Beacon Listener Sample Code**

First parameter in the overloaded constructor of EthernetUDPListener is a delegate (EthernetDataDelegate) called when the beacon arrives from the instrument and the second parameter is the Beacon port on which a UDP listener is started. EthernetDataDelegate bears the following signature and the delegate passed to the constructor is called when the UDP listener receives Beacon data:

```
void BeaconData(EthernetReturnBase ethernetData){}
```

## 5.2 Parsing Beacon Information

EthernetBeaconData (see *EthernetBeaconData Class Diagram*
) contains instrument definitions. The application can obtain complete instrument information by looping through the instrument definitions listed in EthernetBeaconData.

Sample code for looping through the instrument in EthernetDataDelegate passed into the constructor of the EthernetUDPListener:

```
private void BeaconData(EthernetReturnBase ethernetData)
{
      EthernetBeaconData beaconData =
      ethernetData as EthernetBeaconData;
      if (beaconData != null && !string.IsNullOrEmpty(beaconData.IP))
      {
         foreach (InstrumentDefinition deviceInfo in beaconData)
      {
      // Use InstrumentDefiniton for instrument info.
      string instructionSetName = deviceInfo.InstructionSetName;
      string deviceName = deviceInfo.Name.
      }
      }
}
```

**Parsing Beacon Sample Code**

## 5.3 Connecting to the Instrument

Instrument information is obtained from the beacon broadcast.

In order to control the instrument, an application must send a connection request to the instrument to find out what port to connect to for general communication and control of the instrument. The connection request is sent on Admin Port (50185). Upon receipt of the request, the instrument starts a new TCP listener on a port dedicated for the requesting application and responds back with the port number. The port number returned is determined by the device and will be unique for the application. No two applications will be given the same port number on an instrument at the same time.

The RequestPort method exposed by the EthernetSend class allows the application to obtain the communication port from the instrument. Following is the signature of RequestPort method:

```
int RequestPort(string deviceIP, string appName)
int RequestPort(string deviceIP, string appName, string deviceGUID)
```

Three data elements are available from the beacon: 1.) the Device IP address, 2.) the GUID, 3.) the application name requesting the communication port. RequestPort is a synchronous method. RequestPort method parses the response from the instrument and returns the communication port to the caller:

```
EthernetSend ethernetSend = new EthernetSend();
int communicationPort = ethernetSend.RequestPort(
      deviceIPAddress,
      "Application Name",
      deviceGUID);
if (communicationPort > 0)
{
   GilsonTcp tcpSocket = GilsonTcp (
      deviceIPAddress,
      communicationport);

   tcpSocket.SocketClosed += new SocketDelegate(SocketClosed);
   tcpSocket.SocketConnected += new SocketDelegate(SocketConnected);
   tcpSocket.AddListnet(new EthernetDataDelegate(IncomingData));
}
```

**Connecting to Instrument Sample Code**

ethernetSend.RequestPort() connects to the instrument on Admin port (50185) and sends a request for communication port. The instrument sends a response containing the port number the application can connect to. The RequestPort method sends the command to the instrument, processes the response from the instrument, and returns the assigned port number to the calling application.

After a port number has been retrieved from the instrument, the application can connect to the instrument on the assigned port.

```
GilsonTCP tcpSocket = GilsonTcp(deviceIP, port);
```

The code listed above establishes a TCP connection to the instrument on the communication port returned in the call to RequestPort(). The application should establish the TCP connection on this port as quickly as possible. If the application does not establish the TCP connection quickly enough, the device will timeout the connection request and cease listening for a connection on the assigned port.

```
tcpSocket.SocketClosed += new SocketDelegate(SocketClosed);
tcpSocket.SocketConnected += new SocketDelegate(SocketConnected);
```

SocketClosed and SocketConnected delegates exposed by the GilsonTcp class allow notification of socket close and connect. Here is a code fragment passing a callback to GilsonTcp to notify the application when there is incoming data from the instrument:

```
tcpSocket.AddListner(new EthernetDataDelegate(IncomingData));
```

## 5.4 Controlling the Gilson Ethernet Instrument

The instrument can be controlled by an application subsequent to establishing a connection. The instrument is controlled by sending commands specific to the instrument. The device specific commands are listed in the Instruction Set. The Instruction Set is embedded into each instrument. The Instruction Set can be requested from the instrument by sending the 'Get Instruction Set' command to the instrument. Information regarding how to parse the Instruction Set obtained from the instrument is discussed below in *Instrument Instruction Set* section and instrument specific documentation.

The instruction set supports two types of commands:

- Immediate Commands
- Buffered Commands

Buffered commands cause the instrument to perform an action. An application must have a lease or Admin privileges before executing buffered commands.

Immediate commands query the instrument for information. There is no action associated with an immediate command. Applications can successfully execute Immediate commands without a lease or Admin privileges.

## 5.5 Obtaining a Lease for Controlling a Gilson Device

In order to control an instrument an application must complete the following:

- Establish a connection with the instrument
- Become a lessee or administrator of the instrument.

Status commands can be executed before obtaining the lease or becoming an Admin.

An application obtains a lease by sending a RenewLease Command (see *Command Sample XML* ). Only one user application can acquire a lease at any given time.

Lease expiration time is specific to an instrument and is defined within the instruction set. A Lease can be renewed by the lessee via sending the 'Renew Lease' command before the lease expires.

A Lease can be dropped by either by sending 'Drop Lease' command to the instrument or closing the instrument socket connection. If the application closes the TCP socket connection, then the lease is automatically dropped by the instrument. Listed below is sample code for obtaining a Lease:

```
If(tcpSocket.isConnected)
{
     tcpSocket.Send(EthernetMessage.RenewLease);
}
```

**RenewLease Sample Code**

The following is sample code for dropping a lease:

```
If(tcpSocket.isConnected)
{
     tcpSocket.Send(EthernetMessage.DropLease);
}
```

**Drop Lease Sample Code**

## 5.6 Controlling Device as Admin

An application obtains administrative privileges with the device by sending an 'Admin' command. An application successfully completing the 'Admin' command receives all messages to and from the instrument, including messages to all client applications. An admin can execute any command in the instrument's command set. Listed below is sample code for obtaining administrative privileges:

```
If(tcpSocket.isConnected)
{
     tcpSocket.Send(EthernetMessage.SetAsAdmin);
}
```

**Obtaining Admin Privilege Sample Code**

## 5.7 Instrument Instruction Set

The set of commands supported by an instrument is listed in the Instruction Set. The Instruction set for an instrument is obtained via the 'Get Instruction Set' system command. See *EthernetMessages Class* for more information regarding system commands available for controlling Gilson instruments.

Below is sample code to obtain the instruction set from a connected device:

```
if (tcpSocket.IsConnected)
{
   tcpSocket.Send(EthernetMessages.SendRequest);
}
```

## 5.8 Parsing the Instruction Set

The instrument returns the instrument Instruction Set in the response value of the Get Instruction Set command response. Listed below is an XML fragment of the instruction set showing a command definition:

```
<CommandDefinition>
    <CommandName>Collect Data</CommandName>
    <CommandNotes>Turns on data reporting responses</CommandNotes>
    <CommandType>sys</CommandType>
    <CommandExpression>Collect Data</CommandExpression>
    <Parameters>
          <Parameter>
                <ParameterId>0</ParameterId>
                <ParameterName>Channels</ParameterName>
                <ParameterType>String</ParameterType>
                <ParameterNotes>Comma delimited list</ParameterNotes>
    </Parameter>
          <Parameter>
                <ParameterId>1</ParameterId>
                <ParameterName>Event</ParameterName>
                <ParameterType>OnOff</ParameterType>
                <ParameterNotes>Turns events on/off</ParameterNotes>
          </Parameter>
          <Parameter>
                <ParameterId>2</ParameterId>
                <ParameterName>Data</ParameterName>
                <ParameterType>OnOff</ParameterType>
                <ParameterNotes>Turns data on/off</ParameterNotes>
          </Parameter>
    </Parameters>
</CommandDefinition>

<CommandDefinition>
    <CommandName>Get Error</CommandName>
    <CommandType>imd</CommandType>
    <CommandExpression>e</CommandExpression>
    <ReturnExpression>{0}</ReturnExpression>
    <ReturnParameters>
          <ReturnParameter>
                <ParameterId>0</ParameterId>
                <ParameterName>Error</ParameterName>
                <ParameterType>String</ParameterType>
          </ReturnParameter>
    </ReturnParameters>
</CommandDefinition>
```

**Command Definition in Instruction Set Sample XML**

Listed below is sample code for parsing the instruction set. Parsing the instruction set is necessary to access the commands. The code demonstrates extracting the InstructionSetDeviceList from the EthernetResponseData. The InstructionSetDeviceList contains instruction sets for all the instruments defined within the instruction set:

```
private void IncomingData(EthernetReturnBase data)
{
  if (data is EthernetResponseData &&
responseData.CommandName == EthernetMessages.CommandGetInstructionSet)
  {
    InstructionSetDeviceList instructionSetDeviceList =
      new InstructionSetDeviceList();

    instructionSetList.ReadXml(
"<Gilson>" +
responseData.EthernetReturnParameterList["Instruction Set"].ParamValue+
"</Gilson>");

//Get the commands for the specific instrument.
  }
}
```

Below is the code to obtain instructions for a specific instrument. 'instructionSetName' identifies the instruction set obtained from the InstrumentDefinition class. For additional information, refer to the code sample *Parsing Beacon Sample Code*, which demonstrates how to obtain instrument information from beacon data.

```
InstructionSetCommandList commandList =
         instructionSetDeviceList[instructionSetName].Commands;
foreach (InstructionSetCommand command in commandList)
{
   String commandName = command.Name;
}
```

After an application obtains the object InstructionSetCommand, the parameter information is available. Listed below is a code sample which gathers parameter name, type and default values for all the parameters in the parameter list:

```
// Step -1
InstructionSetParameterList parameterList = command.Parameters;
foreach (InstructionSetParameter parameter in parameterList)
{
   // Step - 2
   if (parameter.IsReturnParameter == false)
   {
     string parmeterName = parameter.Name;
     string type = parameter.Units;
      string defaultValue = parameter.DefaultValue.
    }
}
```

In the sample code, the conditional check `if (parameter.IsReturnParameter == false)'` determines if the parameter is a return parameter. The instrument generates return parameters when a command is executed. This code fragment ignores input parameters and extracts return parameters.

Once an application extracts the command then it can build the EthernetCommand object. The EthernetCommand object sends the command to the instrument with the input parameters. Building a command is demonstrated in *Constructing Command Sample Code*.

## 5.9 Sending Commands to the Gilson Ethernet Instrument

Commands are sent to the instrument in the form of an EthernetMessage object (see *EthernetMessage Class Diagram*). Each command is represented by EthernetCommand (see *EthernetCommand Class Diagram*) class. Each EthernetMessage can contain multiple commands.

Each command will generate a separate response after the command is executed by the instrument. Responses are associated with the sequence number assigned to the command.

Below is sample code for building and sending the command to the instrument:

```
// Construct EthernetCommand object and set the parameters.
EthernetCommand command = new EthernetCommand(
      "Move X",
GX-27x,
                                    35,
                                    false);
comamnd.Synchronize = true;
command.Parameters.Add(new EthernetParameter("X Position", "10"));
command.SequenceNumber = 10;

// Construct the EthernetMessage and insert commands into
// EthernetMessage.
EthernetMessage ethernetMsg = new EthernetMessage();
ethernetMsg.IP = devicIPAddress;
ethernetMsg.PortNumber = communicationPort;
ethernetMsg.Commands.Add(command);

//Send EthernetMessage to the instrument.
tcpSocket.TcpSend(ethernetMsg);
```

**Constructing Command Sample Code**

The following is detailed information for the Parameters passed into the constructor of the EthernetCommand:

- **"Move X"**: Command Name.
- **"GX-27x"**: Name of the Instruction Set. This information is obtained from the beacon. This is the property of the InstrumentDefinition(see *InstrumentDefinition Class Diagram*) class.
- **35**: GSIOC unit ID obtained from the beacon.
- **False**: True if command is a synchronized command.

After the EthernetCommand object is constructed, parameters are added to the command using the Parameters property. The EthernetParameter constructor takes two arguments: parameter name and parameter value.

```
EthernetMessage ethernetMsg = new EthernetMessage();
ethernetMsg.IP = devicIPAddress;
ethernetMsg.PortNumber = communicationPort;
ethernetMsg.Commands.Add(command);
```

After constructing the command, it is wrapped inside EthernetMessage class object. EthernetMessage is sent to the instrument by calling TcpSend() method on GilsonTcp socket already connected to the instrument.

Every command sent to the instrument from an application can be assigned a sequence number. The Sequence number sent in the command is returned to the application in the command response generated by the instrument and is used to match up responses with commands. This is done because commands can be executed asynchronously. Because of this, it is possible for responses to commands to be returned in a different order from how they were sent in. The sequence number allows the application to match commands with responses. The sequence number is represented by the SequenceNumber tag in the command XML below:

```
<Gilson>
  <CommandData>
    <Commands>
      <Command>
        <CommandType>Local</CommandType>
        <CommandInfo>
          <InstrumentInfo>
            <DeviceName>GX-27x</DeviceName>
            <DeviceId>35</DeviceId>
          </InstrumentInfo>
          <SequenceNumber>3</SequenceNumber>
          <Synchronize>True</Synchronize>
          <CommandXML>
            <CommandName>Move X</CommandName>
            <Parameters>
              <Parameter>
                <ParameterName>X Position</ParameterName>
                <ParameterValue>10</ParameterValue>
              </Parameter>
            </Parameters>
          </CommandXML>
        </CommandInfo>
      </Command>
    </Commands>
  </CommandData>
</Gilson>
```

**Command Sample XML**

Command parameters are represented by the 'Parameters' tag as shown in the above XML. A command is represented by the class EthernetCommand.

Every command sent to the device is echoed back to all clients connected to the device.

The instrument sends a response for every command. The response includes a response code and response value. The response code indicates success or failure and the response value contains return data.

## 5.10 Synchronous Command

If an application elects to send a command as synchronized, the instrument will send the command response before executing any other commands.

For example, if the 'Move X' command is sent as synchronous, then the response is generated after the X-Motor is finished moving.

The following is sample code setting a command to be synchronized:

```
EthernetCommand command = new EthernetCommand();
comamnd.Synchronize = true;
```

## 5.11 Asynchronous Command

If an application elects to send a command as asynchronous, the instrument will generate a response immediately. The response is generated before the instrument executes the command.

For example, if the 'Move X' command is sent asynchronously, then the response is sent immediately after receiving the command and before the X-Motor finishes moving.

Below is sample code setting a command to be asynchronous:

```
EthernetCommand command = new EthernetCommand();
comamnd.Synchronize = false;
```

## 5.12 System Commands

Every Gilson Ethernet instrument supports System commands. The DeviceName property of a system command is specified as 'System'. Listed below are twelve system commands typically implemented in a Gilson Ethernet device:

1) Get Instruction Set
   - Request the instruction set from the instrument.
2) Renew Lease
   - Request a lease from the instrument. The renewal will be denied if the instrument is already leased by another application. If the lease is not renewed within the timer period specified in the device's instruction set, the lease will time out and be dropped.
3) Drop Lease
   - Drops the lease if the application is either the one with the current lease or is connected as an admin. If the application is not the lease holder or an admin, the request will be denied.
4) Get Firmware Version
   - Returns the version for the Ethernet module and any connected instruments.
5) Kill
   - Drops the specified TCP connection.
6) Reset
   - Restart only the Ethernet module. (This does NOT reset the instrument.)
7) Admin
   - Sets the TCP connection to allow all commands.
8) Get Last Response
   - Request the last few responses sent from the instrument. The number of responses sent depends how it is set up for the device in the system section of the instruction set.
9) Get Client Connections
   - Request a list of all connected user applications.
10) Get Status
   - Request a single status message to be sent immediately.
11) Set NVM
   - Sets the value of the non-volatile memory location specified on the instrument. See instrument documentation.
12) Get NVM
   - Gets the value of the non-volatile memory location specified on the instrument. See instrument documentation.

Below is an Admin command showing the sample XML sent to the instrument. The DeviceName tag is specified as 'System' for all system commands. DeviceId fields are not required for 'System' commands.

```
<Gilson>
  <CommandData>
    <Commands>
      <Command>
        <CommandInfo>
          <InstrumentInfo>
            <DeviceName>System</DeviceName>
            <DeviceId></DeviceId>
          </InstrumentInfo>
          <SequenceNumber>0</SequenceNumber>
          <CommandXML>
            <CommandName>Admin</CommandName>
          </CommandXML>
        </CommandInfo>
      </Command>
    </Commands>
  </CommandData>
</Gilson>
```

**System Command Sample XML**

## 5.13 Response Processing

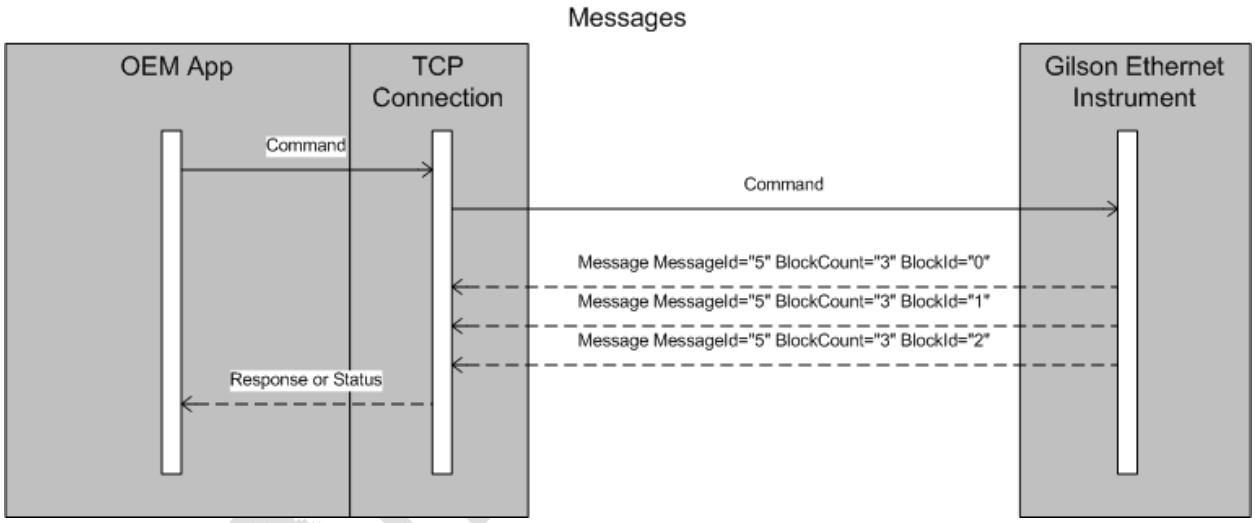
Gilson Ethernet instruments send a response for every command received. The command response is represented by class EthernetResponseData.

All messages sent by the instrument are wrapped in a Message tag. Message tag mechanism allows a single message to be split into multiple packets. The MessageId, BlockCount and BlockId data listed in XML Template below are used to identify multiple packets and assemble the fragments into single XML Document (see *Message Communication from Instrument*).

```
<Message MessageId="5" BlockCount="3" BlockId="0">
<!--Message Content-->
</Message>
```

**Fragmented Response Sample XML**

1) **MessageId**: Identifies a single message. This is the same for all message fragments.
2) **BlockCount:** Identifies the number of fragments into which the message associated with MessageId is split.
3) **BlockId**: Identifies the order of the fragments. BlockId is used to assemble the fragments to construct complete XML message from the instrument.




**Message Communication from Instrument**

The following is a simple response for a 'Move X' command (see *Command Sample XML* ) where the response value is just a string value 'Success':

```
<Gilson>
  <Response>
    <SequenceNumber>3</SequenceNumber>
    <CommandName>Move X</CommandName>
    <Parameters>
      <Parameter>
        <ParameterName>ResponseCode</ParameterName>
        <ParameterValue>2</ParameterValue>
      </Parameter>
      <Parameter>
        <ParameterName>ResponseValue</ParameterName>
        <ParameterValue>Success</ParameterValue>
      </Parameter>
    </Parameters>
  </Response>
</Gilson>
```

**Command Response with Response Value as String Sample XML**

The following is a sample command-response with return parameters from the instrument:

```
<Gilson>
  <CommandData>
    <Commands>
      <Command>
        <CommandType>Local</CommandType>
        <CommandInfo>
          <InstrumentInfo>
            <DeviceName>GX-27x</DeviceName>
            <DeviceId>35</DeviceId>
          </InstrumentInfo>
          <SequenceNumber>3</SequenceNumber>
          <Synchronize>True</Synchronize>
          <Selected>True</Selected>
          <CommandXML>
            <CommandName>Get Z Position</CommandName>
          </CommandXML>
        </CommandInfo>
      </Command>
    </Commands>
  </CommandData>
</Gilson>
```

**Command Expecting Return Parameter Sample XML**

```
<Message MessageId="1886" BlockCount="1" BlockId="0">
  <Gilson>
    <Response>
      <SequenceNumber>3</SequenceNumber>
      <CommandName>Get Z Position</CommandName>
      <Parameters>
        <Parameter>
          <ParameterName>ResponseCode</ParameterName>
          <ParameterValue>2</ParameterValue>
        </Parameter>
        <Parameter>
          <ParameterName>ResponseValue</ParameterName>
          <ParameterValue>
            <ReturnParameters>
              <ReturnParameter>
                <ParameterName>Z Position</ParameterName>
                <ParameterValue>125</ParameterValue>
              </ReturnParameter>
            </ReturnParameters>
          </ParameterValue>
        </Parameter>
      </Parameters>
    </Response>
  </Gilson>
</Message>
```

**Command ResponseData with Response Value Containing Return Parameters Sample XML**

A command response is represented by the class EthernetResponseData. Below is a code sample demonstrating how to access the return parameters:

```
void IncomingData(EthernetReturnBase data)
{
   if (data is EthernetResponseData)
   {
         EthernetResponseData response =
      data as EthernetResponseData

      string parameterName = returnParm.ParamName;
         if(response.EthernetReturnParameterList.Count > 0)
         {
      foreach (EthernetReturnParameter returnParam in
response.EthernetReturnParameterList)
         {
               String parameterValue = returnParam.ParamValue;
            }
         }
      else
      {
      String parameterValue = Response.ResponseValue
      }
      }
      elseif(data is EthernetStatusData)
      {
         // Process status data.
      }
}
```

**Parsing EthernetResponseData Sample Code**

From the code above, IncomingData() is a callback passed for EthernetDataDelegate when the TCP connection is established with the instrument (see *Connecting to Instrument Sample Code* ).

EthernetReturnBase is the base class for all the return data types from the instrument (see *EthernetReturnBase Class Diagram*).

```
if (data is EthernetResponseData)
      EthernetResponseData response = data as EthernetResponseData
```

This code is checking to see if the data returned by the instrument is a response for a command sent.

Here the application checks to see if the return value has a list of return parameters. Either the list of return parameters is retrieved or the single response is retrieved.

```
if(response.EthernetReturnParameterList.Count > 0)
{
foreach (EthernetReturnParameter returnParam in
response.EthernetReturnParameterList)
      {
         String parameterValue = returnParam.ParamValue;
         string parameterName = returnParm.ParamName;
      }
}
else
{
   String parameterValue = Response.ResponseValue
}
```

## 5.14 Status Processing

The EthernetStatusData class represents the status message from the instrument. Status messages can be requested or status messages can be unsolicited.

Status messages contain the following:
1) Full status of the instrument
2) Lease information obtained by the application receiving the status (see *LeaseInfo Class*)
3) All the information associated with the instruments

For all the commands which make the instrument move, the instrument will send a status message which includes only status commands marked as polling during the motion. For example, the 'Home' command causes the instrument to send a status message constantly. Only commands marked as polling are returned while the command is executing.

A status message is sent when the application connects to the device on the communication port.

Status messages can be sent periodically by the instrument. This is defined in the instruction set of the instrument by the field 'Full Status Interval' in the System device section.

An application can solicit a status message from the instrument by sending the instrument the system command 'Get Status'.

Status message XML format:

```
<Gilson>
  <Status>
    <SystemInformation>
      <IPAddress>192.168.0.100</IPAddress>
      <SubnetMask>255.255.255.0</SubnetMask>
      <DefaultGateway>192.168.0.1</DefaultGateway>
      <MAC>00:40:9d:ba:db:ad</MAC>
      <LeaseInformation>
        <LeaseActive>False</LeaseActive>
      </LeaseInformation>
    </SystemInformation>
    <Devices>
      <Device>
        <DeviceName>GX-27x</DeviceName>
        <DeviceId>35</DeviceId>
        <SerialNumber>260H6N080</SerialNumber>
        <Connected>True</Connected>
        <Commands>
          <Command>
            <CommandName>Get XY Position</CommandName>
            <Parameters>
              <Parameter>
                <ParameterName>ResponseCode</ParameterName>
                <ParameterValue>2</ParameterValue>
              </Parameter>
              <Parameter>
                <ParameterName>ResponseValue</ParameterName>
                <ParameterValue>
                  <ReturnParameters>
                    <ReturnParameter>
                      <ParameterName>X Position</ParameterName>
                      <ParameterValue>0</ParameterValue>
                    </ReturnParameter>
                    <ReturnParameter>
                      <ParameterName>Y Position</ParameterName>
                      <ParameterValue>2.3</ParameterValue>
                    </ReturnParameter>
                  </ReturnParameters>
                </ParameterValue>
              </Parameter>
            </Parameters>
          </Command>
        </Commands>
      </Device>
      <Device>
        <DeviceName>406 Single Syringe Pump</DeviceName>
        <DeviceId>7</DeviceId>
        <Connected>False</Connected>
      </Device>
    </Devices>
  </Status>
</Gilson>
```

**Status Message Sample XML**

The following is a code sample for parsing status XML:

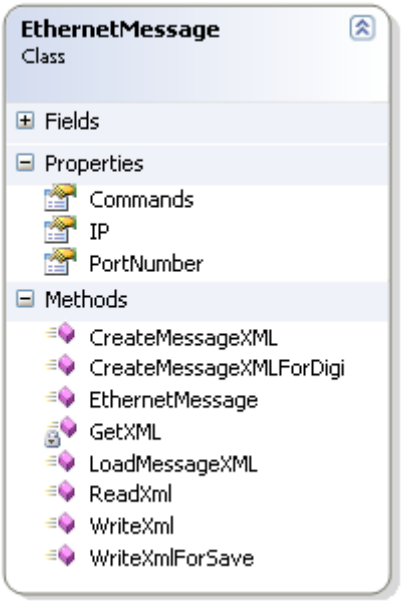
```
void IncomingData(EthernetReturnBase data)
{
   if (data is EthernetStatusData)
   {
         EthernetStatusData statusData =
      data as EthernetStatusData;
         LeaseInfo leaseInfo = statusData.LeaseInfo;
   }
}
```

# 6 Class Definitions

## 6.1 EthernetMessage class

Messages are sent to the instrument using the EthernetMessage class. Commands are represented by the EthernetCommand class. EthernetMessage class object contains the collection of one or more EthernetCommand objects. For a sample command XML, see *Command Definition in Instruction Set Sample XML*

.

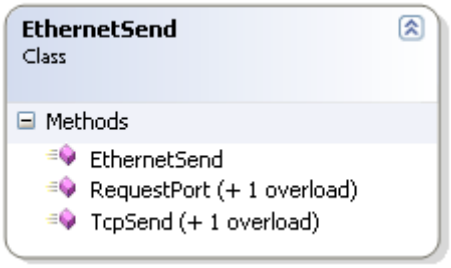



**EthernetMessage Class Diagram**

'Commands' property of the class represents the list of commands. 'IP' & 'Port' class properties represent the IP Address and the communication port of the TCP listener at the instrument.

## 6.2 Ethernet Send

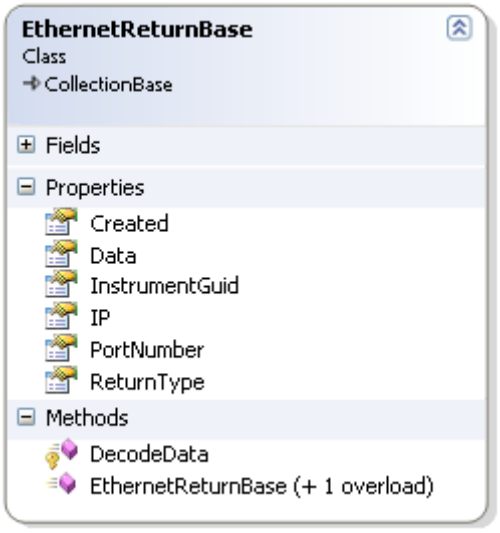
Class EthernetSend is used to obtain the port information from the instrument. Following is the class diagram of EthernetSend class:




**EthernetSend Class Diagram**

## 6.3 EthernetReturnBase Class

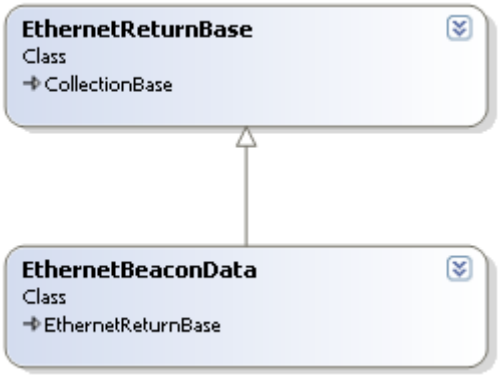
The EthernetReturnBase class represents a response data from the instrument. EthernetReturnBase is the base class for various classes representing data from the instrument (see *Parsing Beacon Sample Code*). Classes representing return data are EthernetBeaconData, EthernetMessage, EthernetStatusData, and EthernetResponseData. The following is the class diagram:




**EthernetReturnBase Class Diagram**

## 6.4 EthernetBeaconData Class

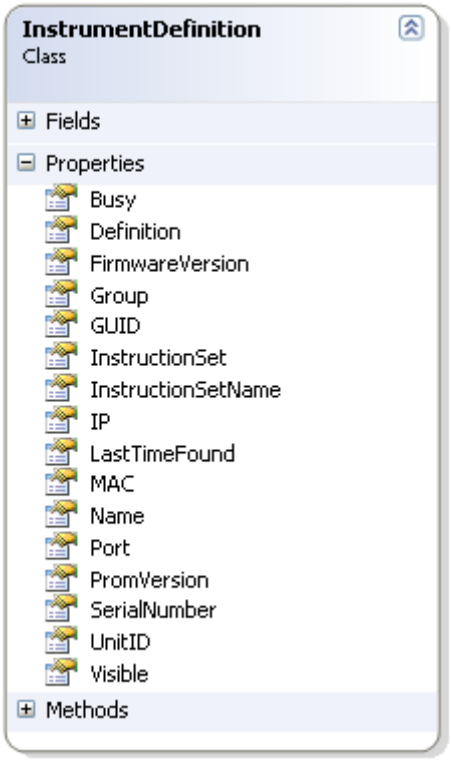
Beacon Data (see *Beacon Data Sample XML*) is represented by the class EthernetBeaconData. Following is the class diagram of the class EthernetBeaconData. This class object contains the collection of instruments obtained from the beacon XML.




**EthernetBeaconData Class Diagram**

## 6.5 InstrumentDefinition Class

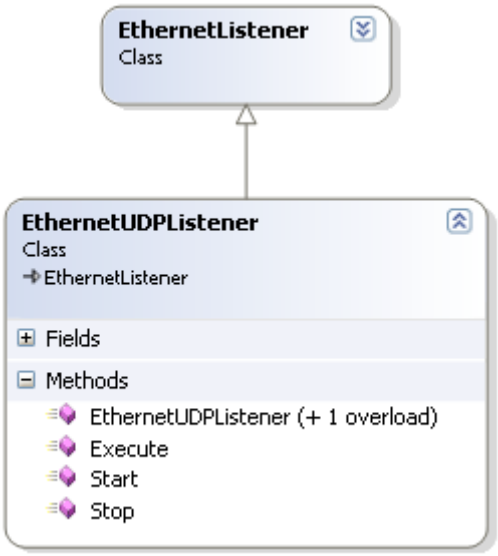
The InstrumentDefinition class represents an Instrument. InstrumentDefinition class encapsulates all the information about the instrument. For code sample on using this class, see *Parsing Beacon Sample Code*. Following is the class diagram of InstrumentDefinition:




**InstrumentDefinition Class Diagram**

## 6.6 EthernetUDPListener Class

EthernetUDPListener class implements UDP Listener. It exposes two methods to start and stop the listener. It provides callbacks to support notification of incoming data. These delegates are of type EthernetDataDelegate defined in Gilson Ethernet assembly (see *Beacon Listener Sample Code* ).

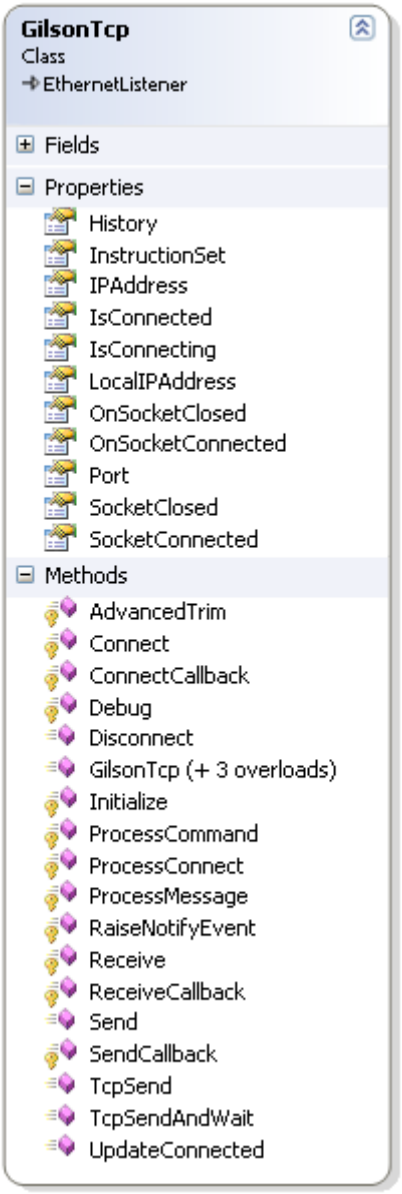



**EthernetUDPListener Class Diagram**

Constructor of this class takes a delegate of type EthernetDataDelegate and port on which to start the UDP listener.

## 6.7 GilsonTcp Class

GilsonTcp class implements TCP client communication using Berkeley sockets. It handles the creation of the socket and establishing a connection to the specified IP address and Port. It exposes methods to send and receive data over a TCP connection. Following is the class diagram:



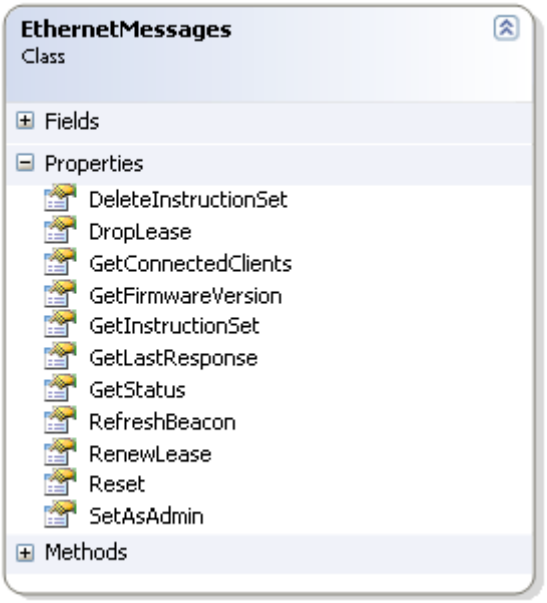

**GilsonTcp Class Diagram**

GilsonTcp class defines delegates supporting notification for socket connection and close. For the sample code regarding use of the class, see *Connecting to Instrument Sample Code*

.

## 6.8 EthernetMessages Class

The EthernetMessages class exposes some of the system commands for controlling Gilson instruments. These commands are general and apply to many Gilson instruments. Listed below is the class diagram of the EthernetMessages class. (All the properties in this class are static.)



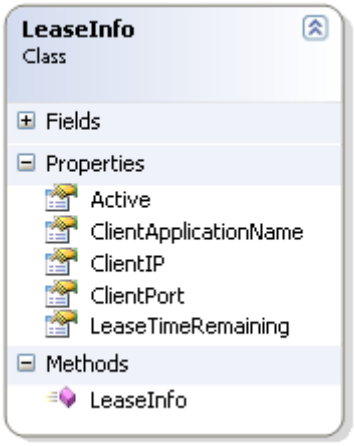

**EthernetMessages Class Diagram**

The following is sample code for using the EthernetMessages class. The sample code sends a command to the instrument to get Admin privileges:

```
if (tcpSocket != null && tcpSocket.IsConnected)
{
tcpSocket.Send(EthernetMessages.SetAsAdmin);
}
```

## 6.9 LeaseInfo Class

This class represents lease information obtained with the instrument. Listed below is the class diagram of the LeaseInfo class:




**LeaseInfo Class Diagram**

Lease information is embedded in the status message sent by the instrument. For information on status message, see *Status Processing*.

The 'Active' property member in LeaseInfo class determines if the lease is active. A Lease must be Active to control the instrument. The duration of lease is configured in the instruction set of the instrument.

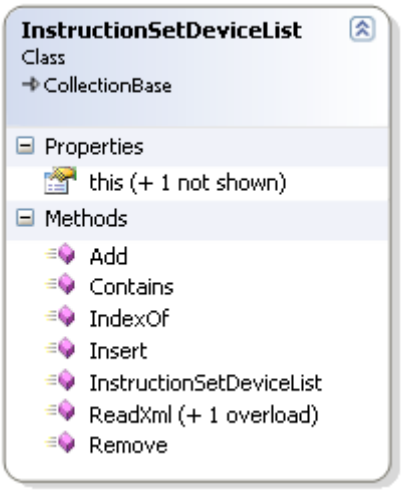
'ClientIP' and 'ClientPort' class properties are the IP Address and Port of the client application connected to the instrument.

'ClientApplicationName' class property is the name of the lessee.

'LeaseTimeRemaining' class property is the time remaining (in seconds) before the lease expires.

## 6.10 InstructionSetDeviceList Class

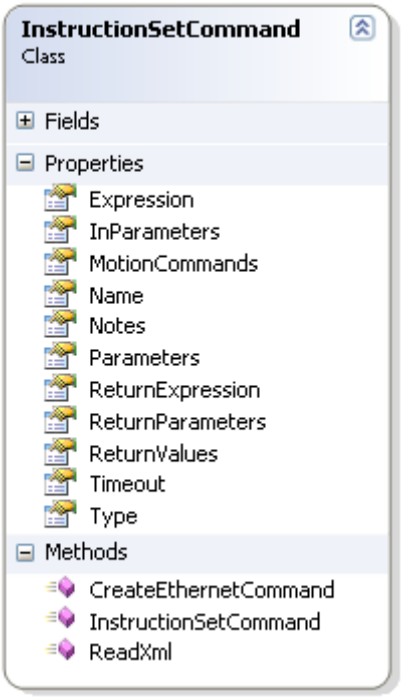
This class represents the list of instruction sets associated with each instrument. Each instrument has an instrument specific command set. Following is the class diagram:




**InstructionSetDeviceList Class Diagram**

## 6.11 InstructionSetCommand Class

This class represents each command in the instruction set. Listed below is the class diagram of the InstructionSetCommand showing its properties:

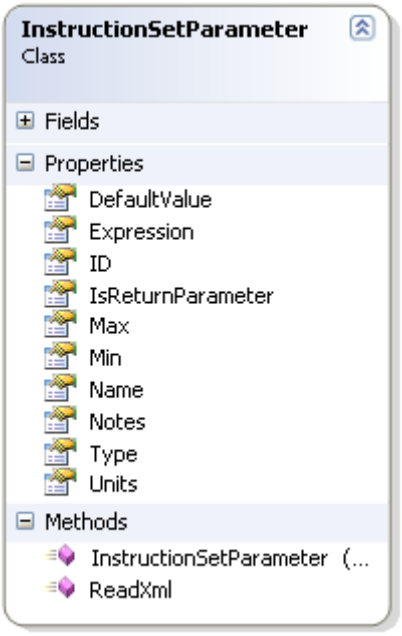



**InstructionSetCommand Class Diagram**

For the sample code showing the usage of the InstructionSetCommand class, see *Parsing the Instruction Set*.

## 6.12 InstructionSetParameter Class

The InstructionSetParameter class represents command parameters from the instruction set. Listed below is the class diagram of InstructionSetParameter:

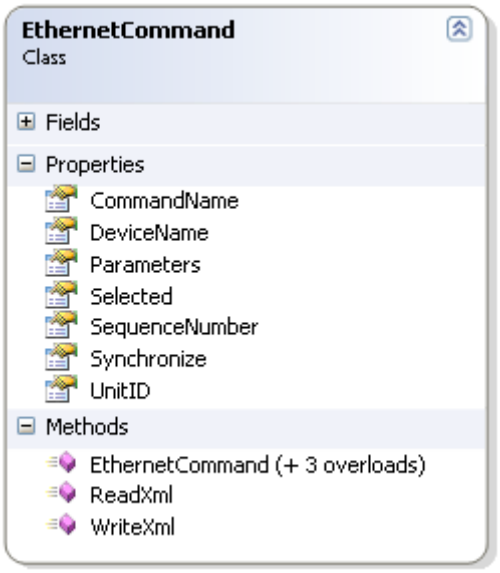



**InstructionSetParameter Class Diagram**

## 6.13 EthernetCommand Class

The EthernetCommand class represents a command. The previous section contains a sample of XML describing a command. (See *Command Sample XML*
) EthernetCommand contains list of parameters represented by EthernetParameter class. The list of parameters associated with a command is accessed using Parameters member property. For sample code on using EthernetCommand class, see *Constructing Command Sample Code*.

Refer to *Command ResponseData with Response Value Containing Return Parameters Sample XML* for sample command XML and associated response XML.

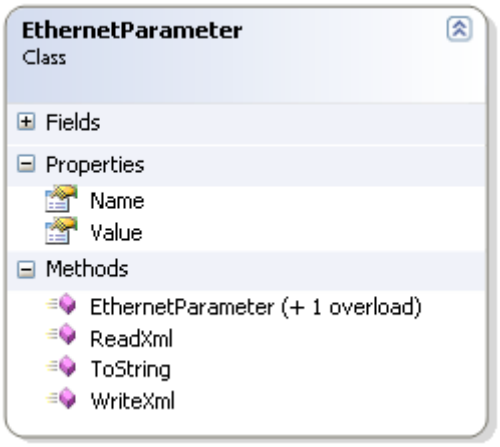



**EthernetCommand Class Diagram**

An application can select to send a command as either synchronous or asynchronous. The synchronous or asynchronous characteristic of a command is identified in the 'Synchronize' property. The EthernetCommand class diagram listed above contains the 'Synchronize' property.

## 6.14 EthernetParameter Class

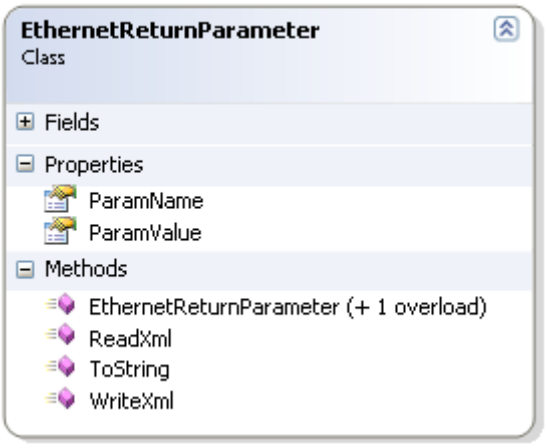
EthernetParameter class represents the parameters associated with the command. A parameter contains both name and the value. The following is the class diagram of the EthernetParameter class:




**EthernetParameter Class Diagram**

## 6.15 EthernetReturnParameter Class

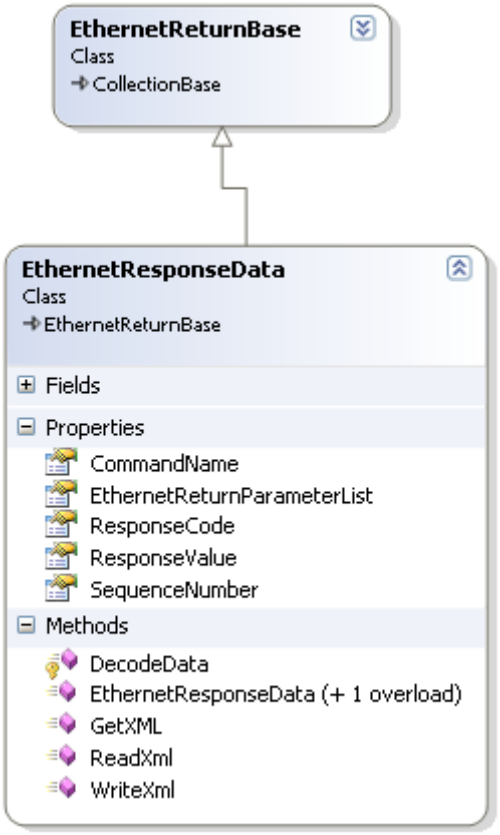
EthernetReturnParameter class represents the return parameter from the instrument. A return parameter can contain the name and value of the parameter. The following is the class diagram of the EthernetReturnParameter class:




**EthernetReturnParameter Class Diagram**

## 6.16 EthernetResponseData Class

EthernetResponseData class represents the response sent by the instrument for a command. Every response contains ResponseCode. ResponseCode signals success or failure and a ResponseValue. The following is the class diagram for EthernetResponseData:




**EthernetResponseData Class Diagram**

The Value associated with ResponseValue is dependent on the command request. ResponseValue types are follows:

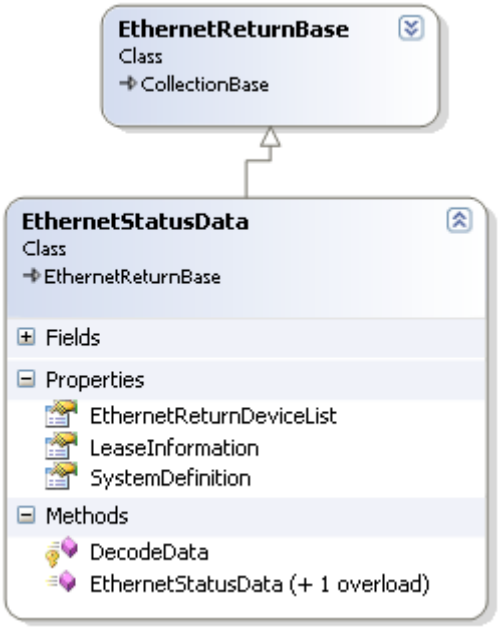
- List of one or more return parameters from the instrument
- String value
- Boolean value True/False
- On/Off value

If the command is expecting a return value, the response value will be return parameters. Return parameters are represented by class EthernetReturnParameter (see *EthernetReturnParameter Class Diagram*).

## 6.17 EthernetStatusData Class

The EthernetStatusData class represents the status message from the instrument. Status messages can be requested or status messages can be unsolicited.

The following is the class diagram of the EthernetStatusData class:




**EthernetStatusData Class Diagram**